End-to-End Data Management Pipeline for a Recommendation System

Final report | DM4ML Assignment 2

Student	Jeet Kumar Singh
BITS ID	2025AG05172
Repository	https://github.com/2025ag05172-gif/recommendation-pipeline

8	5	17	100%	0.0724
pipeline stages	models compared	tests passing	of 1,406 injected defects caught	best NDCG@10 (hybrid)

1. Goal and tools

I have built a pipeline that recommends movies to users from their past ratings and movie genres. It runs eight stages: ingestion, validation, preparation, transformation, feature management, model training, evaluation and orchestration.

Tools and structure

- **Tools:** Python and pandas for data work, pandera for the schema contract, scipy and scikit-learn for the models, MLflow (SQLite backend) for experiment tracking, Prefect for orchestration, parquet and CSV for storage, pytest for 17 tests.
- The code is split into one module per stage under src/, driven by one config file, and every stage reads and writes files so it can be re-run on its own.

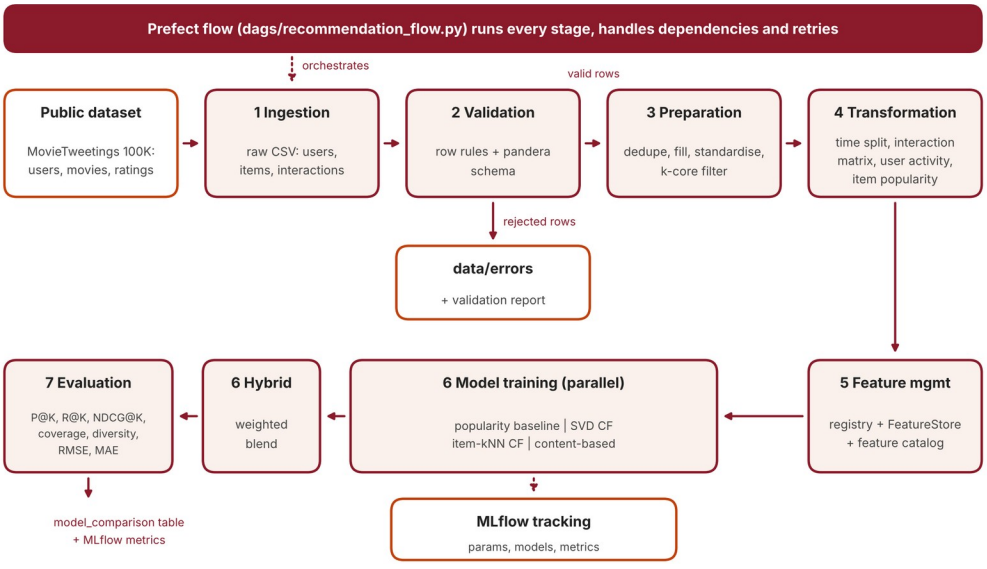


Figure 1. Pipeline architecture (also in the repository: docs/architecture.png)

2. Project structure

Path	What it holds
`configs/config.yaml`	All settings: dataset, thresholds, split, model grids
`src/ingestion/`	Stage 1: download, raw CSVs, logged defect injection
`src/validation/`	Stage 2: row rules, pandera schema, error datasets, report

Path	What it holds
`src/preparation/ src/transformation/`	Stages 3-4: cleaning, time-based split, derived datasets
`src/features/`	Registry, feature builder, FeatureStore (training and prediction)
`src/training/`	Five models, tuning, MLflow tracking
`src/evaluation/`	Metrics, ranking evaluation, results table
`src/pipeline.py`	Plain runner: all stages or one (--stage)
`dags/recommendation_flow.py`	Prefect flow: task dependencies, retries, schedule
`tests/`	17 automated tests
`data/`	raw, interim, processed, features, errors (generated)
`docs/`	Architecture diagram, feature catalog
`reports/`	Validation report, model comparison, tuning tables, this report
`README.md, requirements.txt`	Setup and run steps, pinned dependencies

3. Design decisions

Data

- MovieTweetings 100K snapshot: 16,554 users, 10,507 movies, 100,000 ratings (0-10) from Feb to Sep 2013.
- A rating of 8 or more counts as liked.
- The dataset has no user demographics, so user information is derived from behaviour.
- The raw data is nearly clean, so ingestion adds a small logged set of defects (1,406 rows across several defect types) to prove the validation checks work; this is a config switch and every injected row is listed in a file.

Validation

- Row rules cover missing values, wrong data types, duplicates, ratings outside 0-10, timestamps outside 2010 to now and referential integrity (ratings of unknown users or movies).
- Rows breaking an error rule go to data/errors/ with the reason: 1,200 of 100,200 interactions, 2 users and 3 movies.
- Duplicate ratings and missing genres are warnings that are fixed in preparation.
- The valid rows must then pass a pandera schema, and the run stops if more than 5% of a table is rejected.
- All 1,406 injected defects were caught.

Preparation and transformation

- The latest rating wins for duplicate user-movie pairs, 67 missing genres become 'Unknown', and titles and genres are standardised.
- Users and movies with fewer than 5 ratings are removed (99,000 to 67,006 ratings, 4,298 users, 2,395 movies).
- Because users with one or two ratings cannot be evaluated, this filter was kept even though it is a choice made on the full data.
- The split is by time, not random: ratings before 29 Jul 2013 train (53,604), the most recent 20% test (10,469 after dropping 2,933 rows of users or movies not seen in train).
- Derived datasets are the user activity summary, item popularity and the 4,136 x 2,373 sparse interaction matrix (density 0.55%).

Feature management

- One registry (src/features/registry.py) defines every feature; the feature catalog is generated from it and a test checks the tables match it.
- There are 8 user features (activity count, average rating, rating spread, share liked, days since last interaction, preferred genre and others), a genre profile per user, 8 item features plus 23 genre flags (popularity, average and Bayesian-shrunk rating, age, days since last interaction) and 5 interaction features (rating deviations, recency weight, user-genre affinity).

- All are computed from the train split as of the split date, so nothing from the test period leaks in.
- Training and prediction both read features through the same FeatureStore class; a test confirms the affinity feature is identical with and without ratings.

Models

- Baseline: popularity (rating count).
- Personalised: SVD matrix factorisation (PureSVD on liked items for ranking, bias plus SVD of residuals for rating prediction), item-based collaborative filtering (shrunk cosine similarity), content-based filtering (idf-weighted genre vectors) and a hybrid that blends the four with fixed weights (0.5, 0.25, 0.15, 0.10, not tuned on the test set).
- Factors and shrinkage were chosen on the most recent 15% of train.
- Each model is one MLflow run with parameters, model file, config and later its metrics.

Orchestration

- A Prefect flow (dags/recommendation_flow.py) wraps each stage as a task with explicit dependencies; the four base models train in parallel, the hybrid waits for them, and ingestion retries on network errors.
- It can also run on a daily schedule.
- Seeds are fixed, and a test runs the pipeline twice and compares the result tables.

4. Results

Evaluation uses the 1,892 test users who liked at least one movie in the test period; movies a user already rated are never recommended. K = 10:

Model	Prec@10	Recall@10	NDCG@10	Coverage	Diversity	RMSE	MAE
popularity	0.0253	0.1236	0.0635	0.0131	0.6672	1.5897	1.1970
svd_cf	0.0233	0.1097	0.0650	0.0367	0.6995	**1.4588**	**1.0834**
item_knn_cf	0.0261	0.1219	0.0701	0.2554	0.6841	n/a	n/a
content_based	0.0036	0.0177	0.0108	0.5866	0.1282	n/a	n/a
hybrid	**0.0264**	**0.1263**	**0.0724**	0.0986	0.6733	n/a	n/a

Green = best value in the column (highest for precision, recall and NDCG; lowest for RMSE and MAE). n/a = the model does not predict ratings.

Key findings

- The hybrid is best on precision, recall and NDCG at both K = 5 and K = 10; its NDCG@10 is 0.0724 against 0.0635 for popularity (+14%), while recall@10 is close (0.1263 against 0.1236).
- Item-based CF is second and recommends from a much wider part of the catalog (26% coverage against 1.3% for popularity).
- SVD predicts ratings best (RMSE 1.459 against 1.590 for the popularity baseline).
- The content-based model is weak alone because genres alone separate movies poorly (many movies share the same vector), but it adds coverage inside the hybrid.
- The hybrid trades some coverage (9.9%) for accuracy.

5. Limitations

- The data is small and sparse and covers six months, so every score is low and the differences between models are small; with one test split they should not be read as statistically established.
- Cold-start users and movies are dropped from evaluation, not handled.
- The relevance threshold (rating of 8) and the 5-rating filter are choices that change the numbers.
- The defect injection is synthetic, so the 100% catch rate shows the checks work for those defect types, not that real data has no other problems.
- Features are batch files, not an online store, and the models are only retrained when the flow is run.